拉曼光谱QS值预测模型性能评估报告 - 完整版

报告生成时间: 2026-04-03
数据集: 19,297个拉曼光谱样本, 57个菌株
评估方法: 10折交叉验证
研究阶段: 两阶段完整研究(第一阶段快速比对 + 第二阶段超参数优化)
硬件平台: NVIDIA A100-SXM4-80GB × 8 (云服务器)
训练时间: 第一阶段20分钟 + 第二阶段2小时

研究阶段说明

第一阶段: 模型快速比对与架构筛选 (2026-03-16)

- ✅ 使用经验参数评估CNN和ResNet基线性能
- ✅ 确定ResNet为最优架构(R^2=0.5978, MSE=0.0359)
- ✅ 完成初步性能评估与稳定性分析

第二阶段: 贝叶斯超参数优化 (2026-04-03)

- ✅ 使用Optuna对ResNet进行50次试验优化
- ✅ 获得最优超参数配置
- ✅ 使用最优参数重新训练并保存最佳模型
- ✅ 性能显著提升: R^2从0.598提升至0.835(提升39.7%)

一、方法

1.1 数据集与预处理

1.1.1 数据集描述

本研究使用单细胞拉曼光谱数据集，共包含19,297个光谱样本，涵盖57个不同菌株。每个拉曼光谱样本包含两个通道：拉曼强度(raman intensity)和拉曼位移(raman shift)，每个通道包含2000个数据点。QS值(群体感应强度)作为连续变量进行预测，实际取值范围为[0.09, 1.38]。

数据按照QS值强度分为四类：qs强(强群体感应)、qs中(中等群体感应)、qs低(弱群体感应)和qs无(无群体感应)。这种分类确保了数据集在不同QS值区间的平衡分布，有助于模型的泛化能力评估。

1.1.2 数据预处理

数据预处理流程包括以下步骤：

1. 从原始.fr和.txt文件中提取拉曼光谱数据
2. 将所有光谱统一插值至2000个数据点
3. 构造双通道输入张量 $X \in \mathbb{R}^{N \times C \times L}$，其中 $N = 19,297$ 为样本数量，$C = 2$ 为通道数，$L = 2000$ 为光谱长度
4. 采用Z-score标准化进行归一化处理：

$$X_{\text{norm}} = \frac{X - \mu_{\text{train}}}{\sigma_{\text{train}}}$$

$$y_{\text{norm}} = \frac{y - \mu_{y,\text{train}}}{\sigma_{y,\text{train}}}$$

1.2 模型架构: 1D ResNet

基于第一阶段结果，本研究采用ResNet作为最终模型架构。ResNet通过引入残差连接解决了深层网络的梯度消失问题：

$$\mathcal{H}(x) = \mathcal{F}(x) + x$$

其中 x 表示输入，$\mathcal{F}(x)$ 表示残差函数，$\mathcal{H}(x)$ 表示底层映射。

优化后的ResNet架构:

组件	参数设置	说明
初始卷积层	Conv1d(2→64, kernel=7, stride=1, padding=3)	提取初始特征
最大池化	MaxPool1d(kernel=2)	降采样
残差块数量	3	优化参数
每块滤波器	64×(2^(i//2))	指数增长
Dropout率	0.2078	优化参数

组件	参数设置	说明
全局池化	AdaptiveAvgPool1d(1)	全局特征聚合
全连接层	64→128→64→1	多层感知器

1.3 Optuna超参数优化

1.3.1 优化框架

采用Optuna贝叶斯优化框架，通过Tree-structured Parzen Estimator(TPE)算法高效搜索超参数空间。

1.3.2 优化目标

最小化3折交叉验证的MSE损失：

$$\min_{\theta} \frac{1}{3} \sum_{k=1}^{3} \mathrm{MSE}(D_k^{\mathrm{val}}, f_\theta(D_k^{\mathrm{train}}))$$

1.3.3 超参数搜索空间

超参数	搜索范围	类型
num_blocks	{2, 3, 4}	整数
filters_base	{32, 64, 128}	分类
kernel_size	{3, 5, 7}	分类
dropout_rate	[0.2, 0.5]	浮点数
learning_rate	[1e-5, 1e-3] (对数)	浮点数
batch_size	{128, 256, 512, 1024}	分类

1.3.4 优化过程

- 试验次数: 50
- 并行度: 1 (串行执行)
- 早停策略: ReduceLROnPlateau + EarlyStopping
- 优化时间: 约2小时

1.4 训练策略

1.4.1 损失函数与优化器

- 损失函数: 均方误差(MSE)
- 优化器: Adam(learning_rate=0.000558)
- 学习率调度: ReduceLROnPlateau(factor=0.5, patience=10)

1.4.2 早停策略

监控验证集损失，连续30个epoch无改善则停止训练。

1.4.3 训练配置

- 最大epoch数: 200
- 批大小: 128 (4070 GPU) / 1024 (A100 GPU)
- 交叉验证折数: 10
- 随机种子: 42

1.5 评估方法

1.5.1 交叉验证

采用10折交叉验证确保结果可靠性：

$$CV = \frac{1}{K}\sum_{k=1}^{K}\mathcal{E}(D \setminus D_k, D_k)$$

其中 $K = 10$ 为折数。

1.5.2 评估指标

1. **均方误差(MSE)**: $\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$
2. **均方根误差(RMSE)**: $\sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}$
3. **平均绝对误差(MAE)**: $\frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|$
4. **决定系数(R²)**: $1 - \frac{\sum_{i=1}^{n}(y_i-\hat{y}_i)^2}{\sum_{i=1}^{n}(y_i-\bar{y})^2}$

二、结果

2.1 第一阶段: 基线模型性能对比

表1: 基线模型性能(第一阶段)

模型	MSE	RMSE	MAE	R^2
CNN	0.0389 ± 0.0040	0.1970 ± 0.0104	0.1001 ± 0.0066	0.5635 ± 0.0320
ResNet	**0.0359 ± 0.0025**	**0.1893 ± 0.0067**	**0.0941 ± 0.0037**	**0.5978 ± 0.0310**

结论: ResNet在MSE、RMSE、MAE和R^2四个指标上均优于CNN，R^2达到0.5978，因此选择ResNet进行第二阶段优化。

2.2 第二阶段: Optuna超参数优化结果

2.2.1 最优超参数配置

```
{
  "num_blocks": 3,
  "filters_base": 32,
  "kernel_size": 7,
  "dropout_rate": 0.2077735584403887,
  "learning_rate": 0.0005581372793160988,
  "batch_size": 1024
}
```

2.2.2 优化过程分析

- 最佳MSE: 0.0144 (相比基线0.0359降低59.9%)
- 优化收敛性: 在约30次试验后达到稳定
- 关键发现:
  - 较小的filters_base(32)优于64/128
  - 中等dropout率(0.2078)效果最佳
  - 学习率0.000558为最优值

2.2.3 优化后模型性能

表2: 优化后ResNet性能(第二阶段)

指标	平均值 ± 标准差	相比基线提升
MSE	0.014585 ± 0.002164	**降低59.3%**
RMSE	0.120457 ± 0.009121	**降低36.3%**
MAE	0.059872 ± 0.004978	**降低36.4%**

指标	平均值 ± 标准差	相比基线提升
R^2	0.835106 ± 0.031349	**提升39.7%**

2.3 性能提升分析

表3: 两阶段性能对比

阶段	MSE	RMSE	MAE	R^2	训练时间
第一阶段(基线)	0.0359	0.1893	0.0941	0.5978	20分钟
第二阶段(优化后)	0.0146	0.1205	0.0599	0.8351	74分钟
提升幅度	**-59.3%**	**-36.3%**	**-36.4%**	**+39.7%**	-

2.4 模型稳定性分析

表4: 10折交叉验证稳定性指标(2026-04-03训练)

Fold	MSE	RMSE	MAE	R^2
1	0.01616	0.12713	0.06566	0.79737
2	0.01692	0.13007	0.06354	0.80637
3	0.01738	0.13182	0.06537	0.79305
4	0.01372	0.11715	0.05861	0.85773
5	0.01529	0.12365	0.05983	0.82479
6	0.01262	0.11234	0.05528	0.86488
7	0.01548	0.12440	0.06110	0.82723
8	0.01540	0.12411	0.06354	0.82964
9	0.01122	0.10594	0.05057	0.87985
10	0.01166	0.10797	0.05523	0.87015
最佳Fold	**0.01122**	**0.10594**	**0.05057**	**0.87985**

稳定性指标:

- R^2变异系数: 3.8% (优秀)
- MSE变异系数: 14.8% (良好)
- 最佳R^2与最差R^2差值: 0.0868
- 最佳MSE与最差MSE差值: 0.00616

2.5 模型文件与输出

已保存的最佳模型文件:

1. `best_model_resnet.pth` - 最佳模型权重(包含参数、标准化器等)
2. `best_params_resnet.json` - 最优超参数配置
3. `optuna_history_resnet.png` - Optuna优化历史可视化
4. `training_logs/resnet_training_log_20260403_234627.json` - 最新训练日志
5. `training_logs/resnet_training_results_20260403_234627.png` - 训练可视化图表

模型检查点内容:

```
{
    'model_state_dict': model.state_dict(),
    'model_type': 'resnet',
    'params': params,          # 最优参数
    'scaler_y': scaler_y,      # 标准化器
    'input_shape': X.shape[1:], # 输入形状(2000, 2)
    'best_val_loss': best_val_loss,
    'best_fold': best_fold_idx + 1
}
```

最新训练日志关键信息:

- 训练时间: 2026-04-03 23:46:27
- 整体R^2: 0.836404
- 整体MSE: 0.014585
- 最佳Fold: Fold 9 (R^2=0.87985, MSE=0.01122)

三、讨论

3.1 超参数优化的重要性

本研究表明，系统性的超参数优化对模型性能有决定性影响：

1. **性能提升显著**: R^2从0.598提升至0.835，提升39.7%
2. **关键超参数**:
   - `filters_base=32` 优于64/128，说明模型复杂度并非越高越好
   - `dropout_rate=0.2078` 有效防止过拟合
   - `learning_rate=0.000558` 为最佳学习速率
3. **批次大小影响**: 在A100上使用1024批次，在4070上使用128批次，需根据GPU显存调整

3.2 模型性能的生物学意义

R^2=0.836意味着模型可以解释83.6%的QS值变异，这在生物学应用中具有重要价值：

1. **预测准确性**: RMSE=0.120，MAE=0.061，在实际应用中可接受
2. **应用潜力**: 可用于快速筛选高QS值菌株
3. **研究价值**: 揭示了拉曼光谱与群体感应强度的强相关性

3.3 技术实现优势

1. **端到端学习**: 无需手工特征工程
2. **双通道输入**: 充分利用拉曼强度和位移信息
3. **轻量化设计**: 模型参数量适中，适合部署
4. **完整流程**: 从数据预处理到模型部署的完整解决方案

3.4 局限性及改进方向

当前局限性:

1. 未进行外部独立数据集验证
2. 模型可解释性有待提升
3. 未考虑不同菌株间的生物学差异

未来改进方向:

1. **集成学习**: 结合多种模型进一步提升性能
2. **可解释性分析**: 使用Grad-CAM、SHAP等方法

3. **迁移学习**: 在其他相关数据集上验证泛化能力
4. **实时预测**: 开发在线预测系统

四、结论与建议

4.1 主要结论

1. **模型选择**: ResNet在拉曼光谱QS值预测任务中表现优于CNN
2. **优化效果**: Optuna超参数优化使R^2从0.598提升至0.835(提升39.7%)
3. **预测能力**: 模型可解释83.6%的QS值变异，具有实际应用价值
4. **稳定性**: 10折交叉验证显示模型具有良好的泛化能力(R^2变异系数仅3.8%)

4.2 推荐使用流程

基于本研究的两阶段优化结果，推荐以下使用流程：

```
# 1. 数据准备
python parse_excel_v4.py

# 2. 使用最优参数训练ResNet
python train_pytorch.py --model resnet --quick

# 3. 加载并测试最佳模型
# 代码示例见附录
```

4.3 实际应用建议

1. **硬件配置**: 建议使用至少8GB显存的GPU
2. **批量大小**: 根据GPU显存调整(4070用128，A100用1024)
3. **数据要求**: 新数据需与训练数据相同的预处理流程
4. **模型更新**: 定期使用新数据重新训练以保持性能

4.4 文件清单

文件	用途	重要性
`best_model_resnet.pth`	最佳模型权重	★★★★★

文件	用途	重要性
`best_params_resnet.json`	最优超参数	★★★★★
`train_pytorch.py`	训练脚本	★★★★★
`parse_excel_v4.py`	数据预处理	★★★★
`qs_strain_data.csv`	QS值数据	★★★★
`resnet_results.csv`	详细结果	★★★
`optuna_history_resnet.png`	优化历史	★★

五、附录

A. 模型加载与预测代码

```python
import torch
import numpy as np
from train_pytorch import ResNet1D, RamanDataset

def load_best_model(model_path='best_model_resnet.pth'):
    """加载最佳模型"""
    checkpoint = torch.load(model_path, map_location='cpu')

    # 构建模型
    model = ResNet1D(
        input_length=2000,
        params=checkpoint['params']
    )

    # 加载权重
    model.load_state_dict(checkpoint['model_state_dict'])
    model.eval()

    return model, checkpoint['scaler_y'], checkpoint['params']

def predict_qs_value(model, scaler_y, raman_spectrum):
    """预测单个拉曼光谱的QS值"""
    # 数据预处理(与训练时相同)
    # 注意: 需要实现与训练时相同的标准化逻辑

    with torch.no_grad():
        prediction = model(raman_spectrum)
        # 反标准化
        prediction = scaler_y.inverse_transform(
            prediction.cpu().numpy().reshape(-1, 1)
        )

    return float(prediction[0, 0])
```

B. 10折交叉验证详细结果

表B.1: 优化后ResNet详细结果(2026-04-03训练)

Fold	MSE	RMSE	MAE	R^2	训练轮数
1	0.016161	0.127127	0.065662	0.79737	200
2	0.016919	0.130072	0.063537	0.80637	200
3	0.017377	0.131822	0.065369	0.79305	200
4	0.013724	0.117150	0.058612	0.85773	200
5	0.015289	0.123647	0.059834	0.82479	200
6	0.012620	0.112338	0.055275	0.86488	200
7	0.015475	0.124399	0.061102	0.82723	200
8	0.015398	0.124109	0.063542	0.82964	200
9	0.011218	0.105941	0.050572	0.87985	200
10	0.011655	0.107970	0.055234	0.87015	200
均值	**0.014585**	**0.120457**	**0.059873**	**0.83511**	-

C. 环境配置

```
# requirements.txt
torch>=2.0.0
numpy>=1.24.0
pandas>=2.0.0
scikit-learn>=1.3.0
matplotlib>=3.7.0
optuna>=3.3.0
tqdm>=4.65.0
scipy>=1.11.0
```

D. 快速使用指南

```
# 1. 安装依赖
pip install -r requirements.txt

# 2. 准备数据(已处理则跳过)
python parse_excel_v4.py

# 3. 使用最优参数训练
python train_pytorch.py --model resnet --quick

# 4. 验证模型(新数据)
# 使用附录A中的代码
```

参考文献

[1] He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. In Proceedings of the IEEE conference on computer vision and pattern recognition (pp. 770-778).

[2] Akiba, T., Sano, S., Yanase, T., Ohta, T., & Koyama, M. (2019). Optuna: A next-generation hyperparameter optimization framework. In Proceedings of the 25th ACM SIGKDD international conference on knowledge discovery & data mining (pp. 2623-2631).

[3] Ioffe, S., & Szegedy, C. (2015). Batch normalization: Accelerating deep network training by reducing internal covariate shift. In International conference on machine learning (pp. 448-456). PMLR.

报告完成

作者: 王云龙

研究时间: 2026年3月-4月
研究地点: A100云服务器 + 本地RTX 4070工作站

本报告涵盖了两阶段完整研究过程，提供了从模型选择到超参数优化的完整解决方案，为拉曼光谱QS值预测任务提供了可靠的深度学习模型。